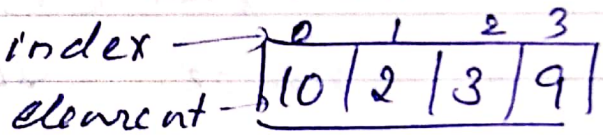
Vectors.

- are much like arrays.
- operations on vector - same big O as in arrays.
- array with dynamic size.
- the size can be increased or decreased as you add or remove elements / grows / to it.



eg: find highest common factor of 2 no.'s.

Can't use array as we don't know how many factors are there for a specific number.
∴ use vectors here.

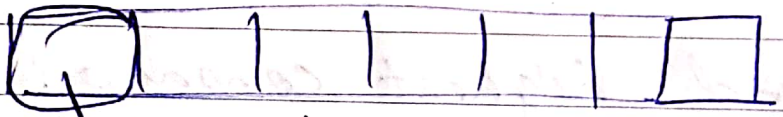
vectors - do not reallocate memory to grow its size, every time a new element is inserted. Instead,

it may allocate some extra storage to accomodate for possible growth.

different libraries - different implementations (for growth) to accomodate possible growth.

Vectors:
Collections of specific type.

vector < double > rainfall (7)
type name. size.



7 elements in this vector

6.3	7.2	9.4	0	4.2	3	1.1
index 0	1	2	3	4	5	6

accessing vectors.

o/p:

rainfall [4]

4.2.

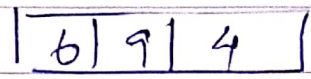
#include <vector>

vector<int> v(3);

v[0] = 6

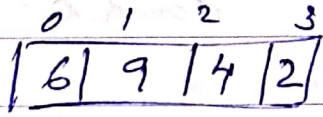
v[1] = 9

v[2] = 4



dynamically add more elements.

v.push_back(2);



cout << v.front(); 6

" << v.back(); 2

" << ~~v[2]~~ v.at(2); 4

printvector(v);

fn: void printvector(vector<int> &

{

```
for (int i=0; i< b.size(); i++)
{
    cout << b[i] << endl;
}
}
```

C++ vector functions

at() → reference to an element

swap() → exchanges the elements between two vectors.

push_back() – adds new element at end.

~~push~~ pop_back() – removes last element from vector

empty() – determines whether vector is empty or not.

insert() — inserts new element at a specified position.

size() — returns the number of elements in the collection.

Hash Tables

data structure, data is stored in array format.

each data value has its own unique index value.

Access of data becomes very fast, if we know the index of the desired data.

22 Sunday 022-343 Week 3

insertions & search operations are very fast, irrespective of the size of the data.

Hash Table

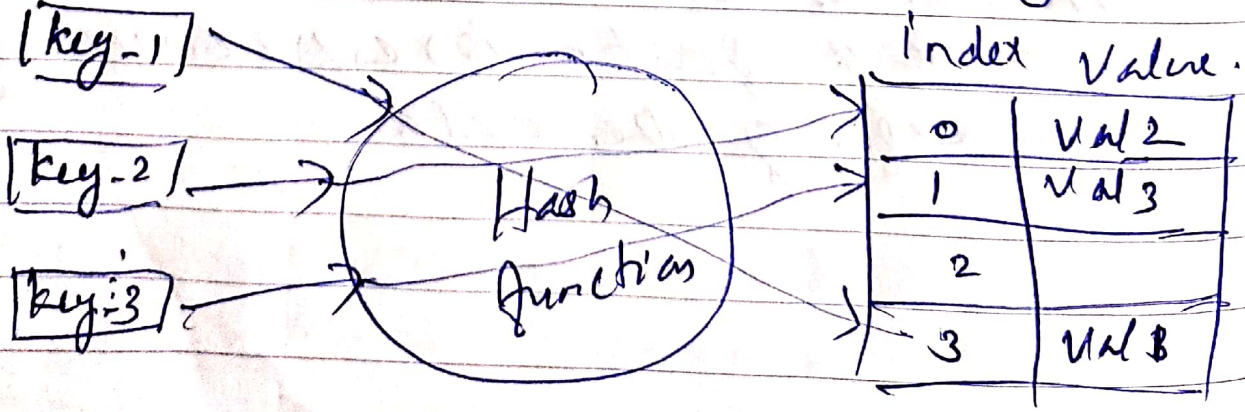
uses array to store.

uses hash functions / technique to generate the index. where the element has to be inserted

Hashing

Technique to convert a range of key values into a range of indexes of an array

eg: hash table of size 10.



key	Hash	Array Index	(key, value)
1	1 % 10 = 1	1	(1, 20)
2	2 % 10 = 2	2	(2, 70)
42	42 % 10 = 2	2	(42, 80)
4	4 % 10 = 4	4	(4, 25)
17	17 % 10 = 7	7	(17, 11)
13	13 % 10 = 3	3	(13, 78)
37	37 % 10 = 7	7	(37, 98)

Collission.

Collission

Linear search O(n)

Binary search (log n

$h(k) = k_i \% m$

↳ size of hash table.

To resolve collisions there are techniques.

Collisions - cannot be removed. It can only be minimized.

1. Open Hashing (closed Addressing)

↓

collisions resolving

method is by Chaining

2. closed Hashing (Open Addressing)

to resolve collisions.

- Linear Probing
- Quadratic Probing
- Double Hashing

Chaining Method / closed addressing

each cell of hash table point to a linked list of records that have the same hash function value.

Example

but requires additional memory outside the table.

Open Addressing

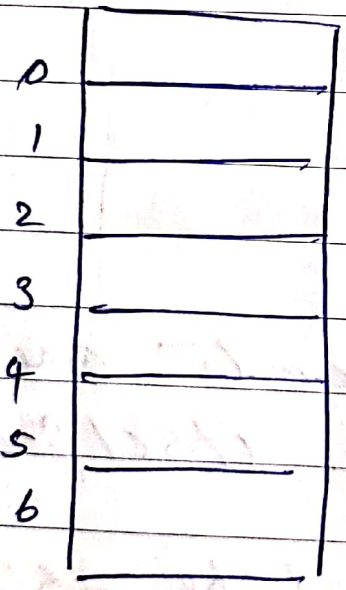
all elements are stored in the hash table itself.

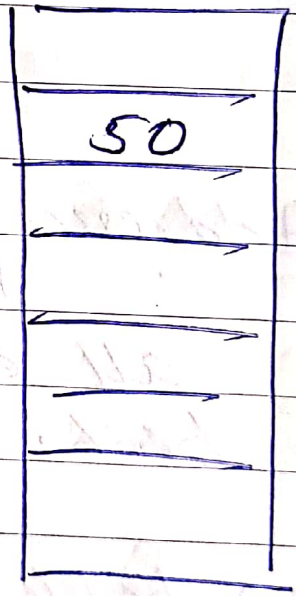
size of table must be greater than or equal to the total number of keys.

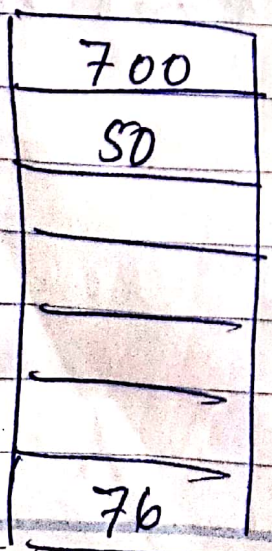
Chaining Technique

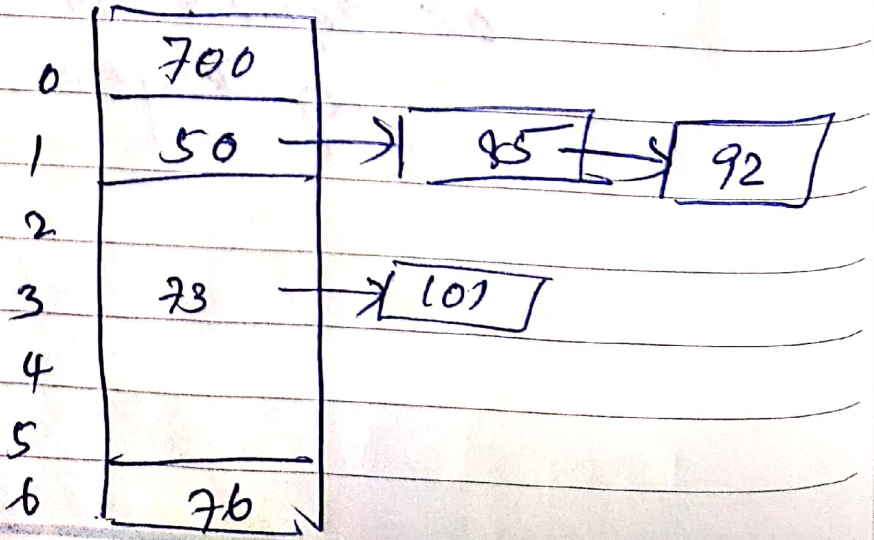
The idea is to make each cell of hash ~~point~~ table point to a linked list of records that have same hash function value.

(key % 7) 50, 700, 76, 85, 92, 73
101









Linear Probing

Open Addressing

insert(k) : keep probing until an empty slot is found. Once empty slot is found, insert k.

search(k) : keep probing until slot's key doesn't become equal to k or an empty slot is reached.

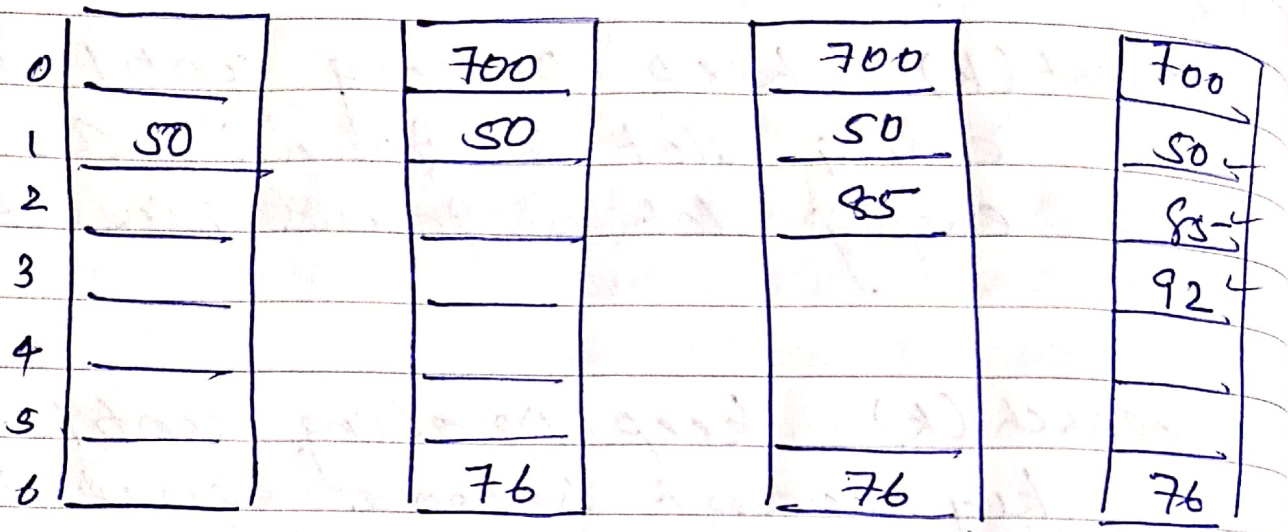
Delete(k) : If we simply, delete a key, search operation may fail. So slots of deleted keys are marked specially as 'deleted'.

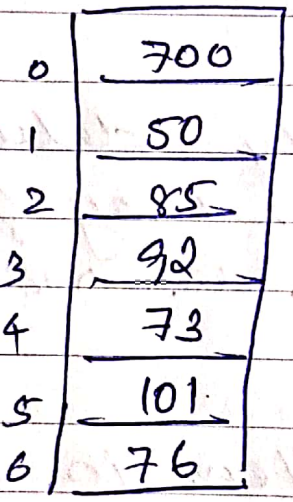
⇒ Insert can insert an item in a deleted slot.
but search doesn't stop at a deleted sl

Linear Probing : Linearly probe for next slot.

(key mod 7)

50, 700, 76, 85, 92, 73, 101





u = (key mod 7)

i = ranges from 0 to n-1

for 92.

$u = 1$.

$(1+0)\%7 = 1$; $(1+1)\%7 = 2$; $(1+2)\%7 = 3$

main problem: takes time to find a free slot or to search an element.

in case collision.

Insert ke at first free location from $(u+i)\%m$ where $i = 0$ to $m-1$

Quadratic Probing

In case of collision;
insert k_i at first free location from
$(u + i^2) \% m$ where $i = 0$ to $m-1$.

(key mod 7)
50, 700, 76, 85, 92, 73,

0	700
1	50
2	
3	
4	
5	
6	76

$85 \% 7 = 1$
u.
$(u + 0^2) \% 7 = 1$
$(u + 1^2) \% 7 = 2$

0	700
1	50
2	85
3	
4	
5	
6	76

$92 \% 7 = 1$
$(1 + 0^2) \% 7 = 1$
$(1 + 1^2) \% 7 = 2.$
$(1 + 2^2) \% 7 = 5$

0	700
1	50
2	85
3	
4	73
5	92
6	76

Double Hashing

insert k_i at first free place from

$(u + v * i) \% m$.

$i = 0$ to $m-1$

v = 2nd hash functions

$$[h_1(k) + h_2(k) * i] \% m$$

$h_1 = k \% 7$

$h_2 = k \% 5$

50, 700, 76, 85, 92, 73

0	700
1	50
2	
3	
4	
5	
6	76

~~$85 \% 7 = 1$~~

~~$85 \% 5 = 0$~~

85 % 7 = 1

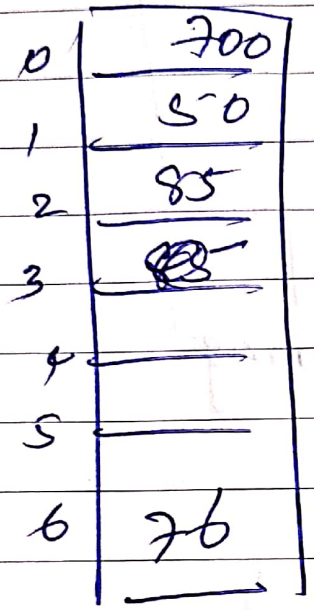
85 % 6 = 1

(1 + 1 * i) % 7 .

(1 + 0) % 7 = 1

(1 + 1) % 7 = 2

(1 + 1 * 2) % 7 = 3



92 % 7 = 1

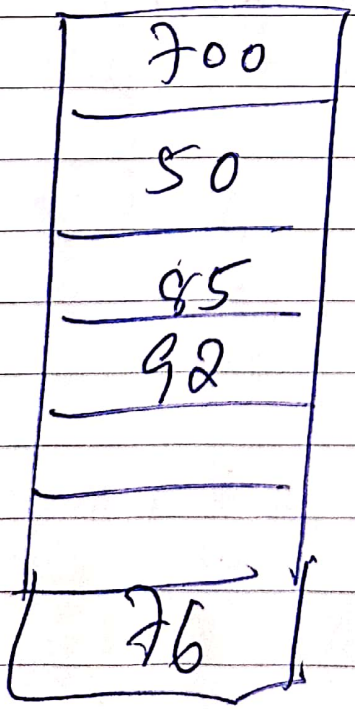
92 % 6 = 2 .

(1 + 2 * i) % 7 .

1 + % 7 = 1

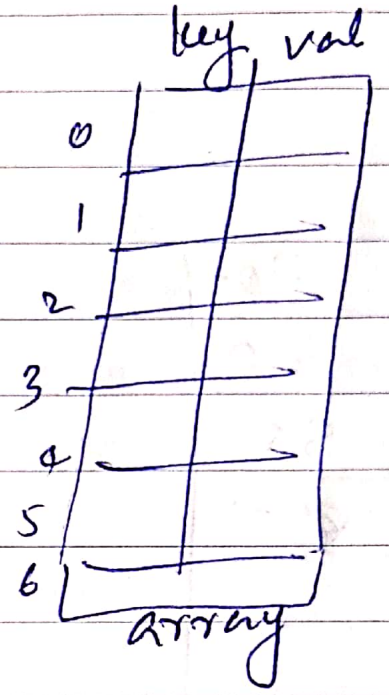
(1 + 2) % 7 = 3

1 +



No. of probes ?

```
struct dataItem
{
    int key
    int val
} * array[10]
```



Division Method
Folding Method
Mid square method.

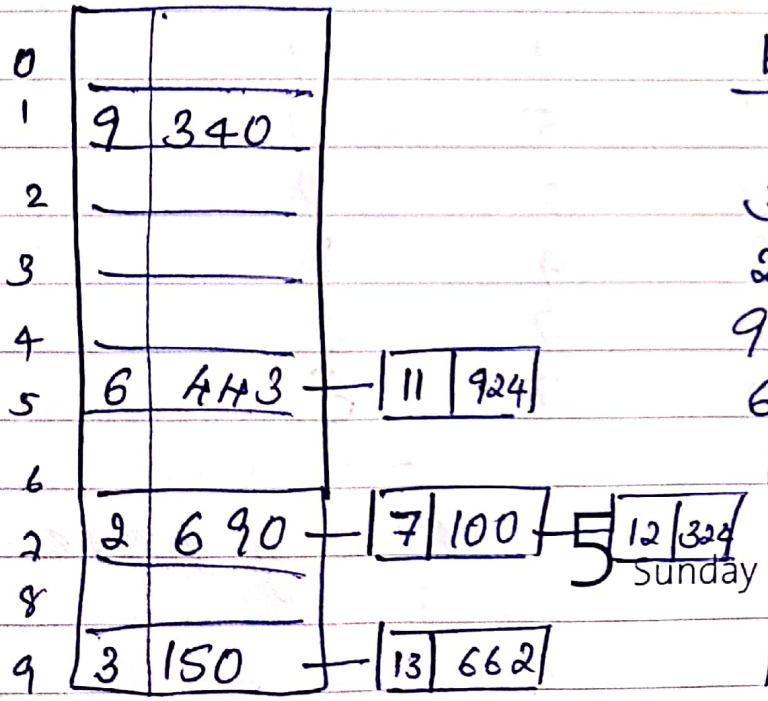
(key, val)

Q) (3, 150), (2, 690), (9, 340), (6, 443), (11, 924), (13, 662), (7, 100), (12, 324).

Use division method & closed addressing to store these values. In case of collision chaining.

$M = 10$.

$h(k) = 2k + 3$.



key	location
3	$(2*3+3)\%10 = 9$
2	$(2*2+3)\%10 = 7$
9	$(2*9+3)\%10 = 1$
6	$(2*6+3)\%10 = 5$
11	$(2*11+3)\%10 = 5$
13	$(2*13+3)\%10 = 9$
7	$(2*7+3)\%10 = 7$
12	$(2*12+3)\%10 = 7$

$k_i \% M$.

$(2k+3)\%M$.

Q)

use division method and open addressing to store these values.

$M = 10$

$h(k) = 2k + 3$.

3, 2, 9, 6, 11, 13, 7, 12.

0	13
1	9
2	12
3	
4	
5	6
6	11
7	2
8	7
9	3

key.	loc.	Probes
3	9	1
2	7	1
9	1	1
6	5	1
11	5 —	2
13	9	2
7	7	2
12.	7	3

Total no. of probes: 12

a) use division method & Quadratic probing to store these values.

$h(k) = 2k + 3$.

$m = 10$.

3, 2, 9, 6, 11, 13, 7, 12.

0	13
1	9
2	12
3	
4	6
5	11
6	2
7	7
8	3
9	

key	location (h)	Probes.
3	$((2*3)+3)\%10 = 9$.	1
2	— 7	1
9	— 1	1
6	— 5	1
11	5	2
13	$((2*13)+3)\%10 = 9$	2
7	7	2
12	7.	5

Total: 15

13

$(9 + 0^2)\%10 = 9$

$(9 + 1^2)\%10 = 0$

7

$(7 + 0^2)\%10 = 7$

$(7 + 1^2)\%10 = 8$

12

$(7 + 0^2)\%10 = 7$

$(7 + 1^2)\%10 = 8$

$(7 + 2^2)\%10 = 1$

$(7 + 3^2)\%10 = 6$

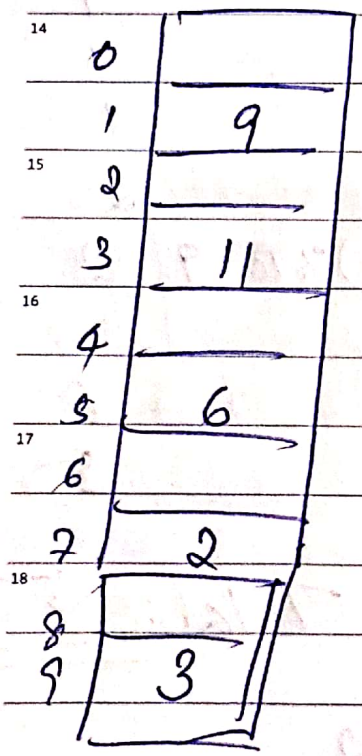
$(7 + 4^2)\%10 = 3$

Use division method & double hashing technique to insert these elements

$h_1(k) = 2k+3$ $\quad m = 10$.

$h_2(k) = 3k+1$

key	loc.(u)	v	probes.
3	9	-	1
2	7	-	1
9	1	-	1
6	5	-	1
11	5	4	3
(13)	9	.	x
(7)	7		x
12	7	7	2



13, 7 → are not able to insert.